In [ ]:
```
#1. Started analysis by importing all libraries, to ensure efficient analysi

#Imported all libraries

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
from vega_datasets import data
import json
import matplotlib.dates as mdates
import polars as pl
import os
import pandas as pd
import geopandas
import plotly.express as px
import plotly.graph_objects as go
import plotly.express as px

# Exported CSV from city website, saved it before importing and used for ana

# Configured Polars to show all columns (default is 10 columns)
pl.Config.set_tbl_cols(15)  # show up to 15 columns
pl.Config.set_tbl_rows(10)  # number of rows to display

Service_Requests_311 = pl.read_csv("/Users/Lavina/Documents/311 data/311_Ser
Service_Requests_311.head()

# Display of data in first 10 rows
print(Service_Requests_311.head(3))
```

In [ ]:
```
#2. Percentage of missing vaues was estimated for the whole data frame
# Total number of rows in the dataframe
total_rows = Service_Requests_311.height

# Calculating percentage of missing values for each column in the DataFrame
missing_percentage = (
    Service_Requests_311
    .select([
        (pl.col(col).is_null().sum() / total_rows * 100).alias(col)
        for col in Service_Requests_311.columns
    ])
)

# Result for each column before cleaning
print(missing_percentage)
```

In [ ]:
```
#3. Optimized different columns for datatype and removed columns not require
#List of columns to parse and cast
colm_date = ['requested_date', 'updated_date', 'closed_date']
colm_float = ['latitude', 'longitude']

# Cleaned dataframe – removed columns that were not of interest
```

```
Service_Requests_311_cleaned = (
    Service_Requests_311
    .drop(['address', 'location_type', 'point'])  # remove unwanted columns
    .with_columns([
        # Parse dates safely and remove time
        pl.col(colm_date).str.strptime(pl.Date, "%Y/%m/%d %I:%M:%S %p", stri

        # Cast float columns
        *[pl.col(col).cast(pl.Float64) for col in colm_float],
    ])
    .unique()  # remove duplicates
)

# Preview first 10 rows
print(Service_Requests_311_cleaned.head(3))
```

In [ ]:
```
# 4. Additional cleaning of data set in polars and converted to pandas for f

#Set higher than total number of columns to ensure all column are displayed
pl.Config.set_tbl_cols(50)

# Total number of rows in the new dataframe
total_rows = Service_Requests_311_cleaned.height

# Created a profiling table to display data of the cleaned dataframe
column_profile = pl.DataFrame({
    "column_name": Service_Requests_311_cleaned.columns,
    "dtype": [str(Service_Requests_311_cleaned[col].dtype) for col in Servic
    "unique_values": [Service_Requests_311_cleaned[col].n_unique() for col i
    "missing_values": [Service_Requests_311_cleaned[col].null_count() for co
    "missing_percentage": [
        round((Service_Requests_311_cleaned[col].null_count() / total_rows)
        for col in Service_Requests_311_cleaned.columns
    ]
})
from IPython.display import display

# Converted the cleaned Polars DataFrame to Pandas for use of different adva
Service_Requests_311_cleaned_pandas = Service_Requests_311_cleaned.to_pandas
)

# Configured Pandas to display all columns
pd.set_option('display.max_columns', None)  # show all columns
pd.set_option('display.width', 200)         # wider display in notebook
pd.set_option('display.max_rows', 10)      # number of rows to show

# Display the DataFrame with updated column names
display(Service_Requests_311_cleaned_pandas)
```

In [ ]:
```
#5 Display unique values for all columns

#Counting unique values for each column to undertstand categorical vs contir
unique_values = Service_Requests_311_cleaned_pandas.nunique()

# Converted to a DataFrame for better display
```

```
unique_values_df = pd.DataFrame({
    "Column": unique_values.index,
    "Unique_Values": unique_values.values
})

# Display the table so all unique value count can be seen
display(unique_values_df)

# 2 Columns have 5 and 4 unique values respectively, this step is performed
columns_to_check = ['Status_description', 'Source']

for col in columns_to_check:
    unique_vals = Service_Requests_311_cleaned_pandas[col].unique()
    print(f"Unique values in '{col}':")
    print(unique_vals)
    print(f"Total unique values: {len(unique_vals)}\n")
```

In [ ]:
```
#6 Further cleaning of DataFrame removing duplicated data in 1 column
# Removed rows where 'Service_description' contains "TO BE DELETED" or "Dupl
Service_Requests_311_cleaned_pandas2 = Service_Requests_311_cleaned_pandas[
    ~(
        Service_Requests_311_cleaned_pandas['Status_description'].str.contai
        | Service_Requests_311_cleaned_pandas['Status_description'].str.uppe
    )
].copy()

# Reset index
Service_Requests_311_cleaned_pandas2.reset_index(drop=True, inplace=True)

# Display cleaned DataFrame
display(Service_Requests_311_cleaned_pandas2)

# Optional: check row counts
print(f"Rows before cleaning: {Service_Requests_311_cleaned_pandas.shape[0]}
print(f"Rows after removing unwanted rows: {Service_Requests_311_cleaned_par
```

In [ ]:
```
#7 If null values were replaced instead of removed
# Original dataframe has been copied to keep raw data
Service_Requests_311_ready_replaced= Service_Requests_311_cleaned_pandas2.cc

# String/object columns, replacing missing values
string_cols = Service_Requests_311_ready_replaced.select_dtypes(include='obj
Service_Requests_311_ready_replaced[string_cols] = Service_Requests_311_reac

# Numeric columns, replacing missing values
numeric_cols = Service_Requests_311_ready_replaced.select_dtypes(include=['f
Service_Requests_311_ready_replaced[numeric_cols] = Service_Requests_311_rea

# Datetime columns, replacing missing values with minimum and maximum
Service_Requests_311_ready_replaced['Updated_date'] = Service_Requests_311_r
    Service_Requests_311_ready_replaced['Updated_date'].min()
)

Service_Requests_311_ready_replaced['Closed_date'] = Service_Requests_311_re
```

```
    Service_Requests_311_ready_replaced['Closed_date'].max()
)

# Reset index
Service_Requests_311_ready_replaced = Service_Requests_311_ready_replaced.re

# Quick check
Service_Requests_311_ready_replaced.info()
```

In [ ]:
```
#8. Most common values for each column
#Dictionary created to store most common values observed
most_common_values = {}

# Looping through all columns to search for most common values
for col in Service_Requests_311_ready_replaced.columns:
    mode_val = Service_Requests_311_ready_replaced[col].mode()
    if not mode_val.empty:
        most_common_values[col] = mode_val[0]
    else:
        most_common_values[col] = None

# For better display, converting to dataframe
most_common_df2 = pd.DataFrame(list(most_common_values.items()), columns=['C

# Pandas settings optimized
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

# Displaying the data
display(most_common_df2)
```

In [ ]:
```
#9. Removed all null values from DataFrame, seemed more informative, less th
# Removed all rows with any null values
Service_Requests_311_ready = Service_Requests_311_cleaned_pandas2.dropna().r

# Step performed to check the shape inorder to understand how many rows were
print(f"Original number of rows: {Service_Requests_311_cleaned_pandas2.shape
print(f"Number of rows after removing nulls: {Service_Requests_311_ready.sha

# Displaying the cleaned dataframe
Service_Requests_311_ready.head(2)
```

In [ ]:
```
#10. Added a column to DataFrame, derived, seemed informative
#Calculate response time for each in days and added to data frame
Service_Requests_311_ready['Response_days'] = (
    Service_Requests_311_ready['Closed_date'] -
    Service_Requests_311_ready['Requested_date']
).dt.days
```

In [ ]:
```
#11.Further optimization of DataFrame, Service_name optimized to ensure corr
#Copied original DataFrame
Service_RequestsG_311_ready = Service_Requests_311_ready.copy()

# Checked for missing Service_name
service_names = Service_RequestsG_311_ready['Service_name'].fillna('Other')
```

```
# Created a default Service_Group = first word (vectorized)
Service_RequestsG_311_ready['Service_Group'] = service_names.str.split().str

# Mask for Service_name starting with Z or z as Z seemed random
mask_z = service_names.str.startswith(('Z', 'z'))

# Vectorized extraction for Z/z-prefixed names
# Remove leading Z/z and any dash/space
z_names_clean = service_names[mask_z].str.lstrip('Zz').str.lstrip(' -')

# Used the first meaningful word (split by space, then split by dash)
z_group = z_names_clean.str.split().str[0].str.split('-').str[0]

# Updated Service_Group for Z/z rows
Service_RequestsG_311_ready.loc[mask_z, 'Service_Group'] = z_group

# Saved new DataFrame as CSV in Documents folder
documents_path = os.path.join(os.path.expanduser('~'), 'Documents', 'Service
Service_RequestsG_311_ready.to_csv(documents_path, index=False)

print(f"Saved CSV to: {documents_path}")

# Step 7: Display first 3 rows
Service_RequestsG_311_ready.head(3)
```

In [ ]:
```
#11 Detect object or category dtype columns
desired_categorical_cols = Service_RequestsG_311_ready.select_dtypes(include
("Categorical columns:", desired_categorical_cols.tolist())
display(Service_RequestsG_311_ready[desired_categorical_cols].describe())
display(Service_RequestsG_311_ready.describe())
```

In [ ]:
```
#12. Most common values for all columns of cleaned DataFrame
#Dictionary created to store most common values
most_common_values = {}

# Looping through all columns
for col in Service_RequestsG_311_ready.columns:
    mode_val = Service_RequestsG_311_ready[col].mode()
    if not mode_val.empty:
        most_common_values[col] = mode_val[0]
    else:
        most_common_values[col] = None

# Converting to dataframe for better display
most_common_df = pd.DataFrame(list(most_common_values.items()), columns=['Co

# Changing Pandas settings for the session
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

# Display data
display(most_common_df)
```

In [ ]:
```
#13Estimation of total number open and closed complaints by different Servic
#Data required for 2 categories
df = Service_RequestsG_311_ready[['Service_Group', 'Status_description']]

# Grouping by Service_Group and Status_description
status_counts = df.groupby(['Service_Group', 'Status_description']).size().u

# Added total columns, sort performed by total complaints
status_counts['Total'] = status_counts.sum(axis=1)
status_counts_sorted = status_counts.sort_values(by='Total', ascending=False

# Display all Service_Groups by total complaints
print(status_counts_sorted.head(20))
```

In [ ]:
```
#14. Estimation by Agencies Counting number of complaints handled by agencie
Agency_counts = Service_RequestsG_311_ready['Agency_responsible'].value_cour

# Agency that receives maximum complaints
max_agency_responsible_count = Agency_counts.idxmax()
max_agency_responsible = Agency_counts.max()

print(f"Agency with maximum complaints: {max_agency_responsible_count}")
print(f"Number of complaints: {max_agency_responsible}")

top_10_agencies = Agency_counts.head(10)
display(top_10_agencies)
```

In [ ]:
```
#15. Exploring different parameteres in DataFrame to understand response tim
#Check differfent parameters
Service_RequestsG_311_ready[['Service_Group', 'Response_days','Comm_name']].
# All the services are grouped and the response time statistics have been ca
Summary_service_responsetime = Service_RequestsG_311_ready.groupby('Service_
    count='count',
    mean='mean',
    median='median',
    min='min',
    max='max'
).reset_index()

# Sorting values by mean response time in days.
Summary_service_responsetime = Summary_service_responsetime.sort_values(by='

# Displaying top 10 services
top_10_services = Summary_service_responsetime.head(10)
display(top_10_services)
```

In [ ]:
```
#16. Estimation in percentage of complaints received according to different
#Count of complaints per Service_Group
counts = Service_RequestsG_311_ready['Service_Group'].value_counts()

# Percentages calculated
percentages = counts / counts.sum() * 100

# Separating groups in big and small groups
```

```
big_groups = counts[percentages >= 5]
small_groups_sum = counts[percentages < 5].sum()

# Combined into final series using pd.concat
counts_combined = pd.concat([big_groups, pd.Series({'Other': small_groups_su

# Pie chart prepared
plt.figure(figsize=(8,8))
plt.pie(
    counts_combined,
    labels=counts_combined.index,
    autopct='%1.1f%%',
    startangle=140,
    colors=sns.color_palette('tab10', len(counts_combined))
)
plt.title('Complaints received per Service Group (groups <5% combined as Oth
plt.show()
```

In [ ]:

```
# Prepare DataFrame ----
df = Service_RequestsG_311_ready[['Service_Group', 'Comm_name']].copy()
df['Comm_name'] = df['Comm_name'].astype('category')
df['Service_Group'] = df['Service_Group'].astype('category')

# Exploring data to understand top service group per neighborhood
top_sg = df.groupby(['Comm_name', 'Service_Group'], observed=True).size().re
# select the service group with max requests for each neighborhood
top_sg = top_sg.loc[top_sg.groupby('Comm_name')['request_count'].idxmax()].r

# Loading GeoJSON
geojson_path = "/Users/Lavina/Documents/311 data/Comm_Bound.geojson"
with open(geojson_path) as f:
    geojson_data = json.load(f)

comm_col = 'name'

# Clean name
top_sg['Comm_name'] = top_sg['Comm_name'].str.strip()
for f in geojson_data['features']:
    f['properties'][comm_col] = f['properties'][comm_col].strip()

# Mapped service groups to numeric values
service_groups = top_sg['Service_Group'].unique().tolist()
sg_to_num = {sg: i for i, sg in enumerate(service_groups)}
num_to_sg = {i: sg for sg, i in sg_to_num.items()}
top_sg['sg_num'] = top_sg['Service_Group'].map(sg_to_num)

# Assign colorscale : magenta
magenta_palette = ['#ffb3d9', '#ff66b3', '#ff1a75', '#99004d', '#cc0066']  #
colorscale = [[i/(len(service_groups)-1), magenta_palette[i % len(magenta_pa
              for i in range(len(service_groups))]

# Created choropleth map to visualize the data for the whole DataFrame
fig = go.Figure(go.Choropleth(
    geojson=geojson_data,
    locations=top_sg['Comm_name'],
    z=top_sg['sg_num'],
```

```
    featureidkey=f"properties.{comm_col}",
    colorscale=colorscale,
    colorbar=dict(
        title="Service Group",
        tickvals=list(sg_to_num.values()),
        ticktext=list(sg_to_num.keys())
    ),
    marker_line_width=0.5,
    marker_line_color='white'
))

# Displaying in Hover text as significant amount of data is displayed
fig.update_traces(
    hovertemplate="<b>%{location}</b><br>Top Service Group: %{customdata}<ex
    customdata=top_sg['Service_Group']
)

# Layout of the graph
fig.update_layout(
    title_text="311 Complaints - Top Service Group per Neighborhood",
    geo=dict(fitbounds="locations", visible=False),
    width=1400,
    height=900
)

fig.show()
```

In [ ]:

```
#Selected categorical columns, graphs plotted
categorical_columns = Service_RequestsG_311_ready.select_dtypes(
    include=['object', 'category']
).columns

# Subplot grid
n_cols = 3
n_rows = int(np.ceil(len(categorical_columns) / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(18, 5*n_rows))

# Ensured the axes is 1D for easy iteration
axes = axes.flatten() if len(categorical_columns) > 1 else [axes]

for i, col in enumerate(categorical_columns):
    # Only compute top 15 counts on the fly, no extra copy
    counts = Service_RequestsG_311_ready[col].value_counts().nlargest(15)

    axes[i].barh(counts.index.astype(str), counts.values, color='#006400', a
    axes[i].set_title(col)
    axes[i].set_xlabel("Count")
    axes[i].invert_yaxis()  # Highest count on top

# Remove any unused subplots
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
```

```
plt.suptitle("Figure 2: Categorical Parameters (Bar Plots)", y=1.02)
plt.show()
```

In [ ]:
```
# Required columns selected
df = Service_RequestsG_311_ready[['Service_Group', 'Response_days']].copy()

# Optimize dtypes
df['Service_Group'] = df['Service_Group'].astype('category')
df['Response_days'] = df['Response_days'].astype('float32')

# Pre-aggregate box plot statistics
agg = df.groupby('Service_Group', observed=True)['Response_days'].agg(
    min='min',
    q1=lambda x: x.quantile(0.25),
    median='median',
    q3=lambda x: x.quantile(0.75),
    max='max'
).reset_index()

# Selection of a color palette
service_groups = df['Service_Group'].cat.categories.tolist()
palette = px.colors.qualitative.Set2  # distinct colors per group
sg_to_color = {sg: palette[i % len(palette)] for i, sg in enumerate(service_

# Created box plot
fig = go.Figure()

for sg in service_groups:
    df_sg = agg[agg['Service_Group'] == sg]
    fig.add_trace(go.Box(
        x=[sg] * len(df_sg),  # single x-position per Service_Group
        lowerfence=df_sg['min'],
        q1=df_sg['q1'],
        median=df_sg['median'],
        q3=df_sg['q3'],
        upperfence=df_sg['max'],
        name=sg,
        marker_color=sg_to_color[sg],
        boxpoints=False  # skip individual points to save memory
    ))

# Layout improvements
fig.update_layout(
    title="Response Time by Service Group ",
    xaxis_title="Service Group",
    yaxis_title="Response Days",
    width=1000,
    height=600,
    plot_bgcolor="#F5F5F5",
    paper_bgcolor="white"
)

fig.show()
```

In [ ]:
```
# Only necessary columns kept
df = Service_RequestsG_311_ready[['Service_Group', 'Response_days']].copy()

# Optimized dtypes
df['Service_Group'] = df['Service_Group'].astype('category')
df['Response_days'] = df['Response_days'].astype('float32')

# Count complaints per Service_Group in descending order
service_counts = df['Service_Group'].value_counts().sort_values(ascending=Fa
sorted_groups = service_counts.index.tolist()  # highest to lowest complaint

# Pre-aggregate box plot stats
agg = df.groupby('Service_Group', observed=True)['Response_days'].agg(
    min='min',
    q1=lambda x: x.quantile(0.25),
    median='median',
    q3=lambda x: x.quantile(0.75),
    max='max'
).reindex(sorted_groups).reset_index()  # reorder by complaint count

# Defined colors (distinct per Service_Group)
palette = px.colors.qualitative.Set2
sg_to_color = {sg: palette[i % len(palette)] for i, sg in enumerate(sorted_g

# Create box plot
fig = go.Figure()

for sg in sorted_groups:
    df_sg = agg[agg['Service_Group'] == sg]
    fig.add_trace(go.Box(
        x=[sg] * len(df_sg),  # x-position
        lowerfence=df_sg['min'],
        q1=df_sg['q1'],
        median=df_sg['median'],
        q3=df_sg['q3'],
        upperfence=df_sg['max'],
        name=sg,
        marker_color=sg_to_color[sg],
        boxpoints=False  # avoid plotting all points
    ))

# Layout
fig.update_layout(
    title="Response Time by Service Group (Sorted by Complaint Volume)",
    xaxis_title="Service Group",
    yaxis_title="Response time in days",
    width=1200,
    height=600,
    plot_bgcolor="#F5F5F5",
    paper_bgcolor="white"
)

fig.show()
```

Lavina D

Lavina D

In [ ]: